DISHA: how our ML model works

A short, plain-language brief for the team, and what to say to the judges.

1. The idea in one paragraph

Our model reads a student's **academic record** and estimates the chance that the student will **drop out**. It learned this by studying thousands of past students whose final outcome we already know. When the chance is high enough that we are confident, it raises a **signal** for a mentor, together with the reasons in plain words. The model never acts on its own: a person always reviews the signal and decides what to do. That is our rule: **No Alert Without Action**.

2. The data we used, and why

Item	What it is
Dataset	UCI *Predict Students' Dropout and Academic Success*: a public research dataset
Source	Real, anonymised records from one higher-education institution in Portugal
Size	4,424 students. Final outcome known: 2,209 graduated, 1,421 dropped out, 794 still enrolled late
In our repo	`ml/data/processed/data.csv` (cleaned); the original is in `ml/data/raw/`

Why this dataset?

- It is **real**, public and free to use, and it has the **final outcome** for every student. A model can only learn to predict dropout from students whose outcome is already known.
- It tracks the same student **semester by semester** (subjects taken, passed, exams sat, grades), which is exactly what an early-warning system needs.
- We searched for an Indian equivalent. No public Indian dataset has student-level semester results *and* a final dropout outcome; the real Indian data sits with universities and researchers.

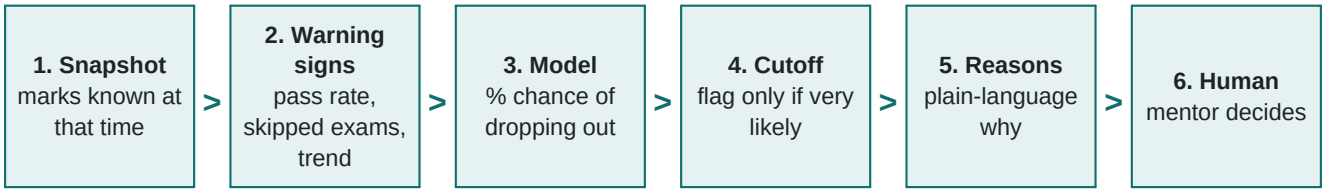
What we kept, and what we left out

Group	Columns	Used how
Kept as model inputs (15)	Grade before university, admission grade, course; and for each semester: subjects enrolled, credited, exams sat, subjects passed, no-exam subjects, average grade	The model learns from these
Kept aside (4)	Gender, age, displaced, special needs	**Never** fed to the model. Used only to check the model is fair to every group
Dropped (17)	Family background, marital status, nationality, economy figures, fees and scholarship	Not academic. Our product focuses on academic data only

Cleaning we did

- Fixed broken column headers (a hidden byte at the start, a stray tab, a Portuguese spelling).
- 180 students, all in one course, have every semester field recorded as 0, even though 75 of them graduated. That is missing paperwork, not students who skipped. We mark those semesters as **missing** instead of treating them as zero.
- 3 students listed more skipped exams than subjects taken; we capped the number. No student was deleted or invented.

3. How it works, step by step



Step 1: Take a snapshot at the right moment

The model runs at three points, and each time it only sees what was known at that moment. It never peeks at results that have not happened yet (that would be cheating, called *data leakage*).

Checkpoint	When	What it sees
cp0	At admission	Grades before university, course
cp1 (main one)	After semester 1	+ semester 1 results
cp2	After semester 2	+ semester 2 results and the change between semesters

Step 2: Turn raw marks into warning signs

Raw numbers are turned into signs that mean the same thing in every course, for example:

- **Pass rate**: subjects passed out of subjects taken (2 of 6 = 33%).
- **Skipped exams**: share of subjects where no exam was taken.
- **Failed subjects**: how many were not passed (a backlog building up).
- **Average grade**, and at cp2 the **trend**: getting better or worse.

Step 3: The model gives a percentage

We use **gradient boosting**. Picture up to a few hundred small yes/no question trees ("Did they pass under half their subjects?" "Did they skip exams?"). Each new tree fixes the mistakes of the ones before it. Together they turn a student's warning signs into one number: the chance of dropping out. It learned this pattern from about 3,500 past students.

Step 4: Make the percentage honest (calibration)

We adjust the scores so they match reality: when the model says **80%**, about 8 out of 10 such students really did drop out. Without this, a cutoff would mean nothing.

Step 5: Only raise a signal when we are confident (the cutoff)

This is how we avoid false alarms. We pick the cutoff so that **at least 85% of flagged students really are at risk**, checked on students the model never saw. Then:

Level	When	Result
HIGH	Chance above the 85%-sure cutoff	Signal raised
MEDIUM	Somewhat lower chance AND a clear academic rule agrees (e.g. passed under half)	Signal raised
WATCH	Some risk, not enough to alert	Listed for staff, no signal
ON TRACK	Low chance	Nothing
NO DATA	Semester record missing	Use the admission check instead

Capacity cap: at most **15%** of students get a signal, so mentors can follow up on every one.

Step 6: Give reasons, and let a human decide

Every signal comes with reasons a mentor can check, e.g. *"passed 2 of 6 subjects; no exam taken in 3 subjects; main factor: exams skipped in semester 1"*. The mentor approves, changes or dismisses it.

4. Worked example (made-up student)

> After semester 1, a student took **6** subjects, sat exams in **3**, passed **2**, average grade 10.8 out of 20.
> **Warning signs:** pass rate 33%, half the exams skipped, 4 subjects not passed.
> **Model:** 81% chance of dropping out, which is above the HIGH cutoff, so it becomes a **HIGH signal**.
> **Mentor sees:** "High risk (81%). Passed 2 of 6 subjects; no exam in 3 subjects; 4 not passed." The mentor approves, and a support case with an owner and a deadline is opened.

5. How well it works (real results)

Tested on **851 students the model never saw** during training (20% kept aside). About 31% of them actually dropped out.

Checkpoint	Flagged	Of those flagged, really dropped out	Of all dropouts, how many we caught	Ready to use?
cp0: admission	2%	65%	5%	No (too weak)
cp1: after sem 1	15%	**94.5%**	46%	**Yes**
cp2: after sem 2	15%	**97.6%**	47%	**Yes**

In plain words (cp1): after the first semester we flag about 1 in 7 students. Roughly **19 out of every 20** flags are correct. We catch almost half of all future dropouts with those flags; more sit in the WATCH list. Among students we put ON TRACK, only 5% dropped out.

Better than simple rules: we compared against plain rules ("passed under half", "skipped an exam") and against a simpler model. Our model ranked at-risk students better than both (score 0.84 vs 0.67 for rules and 0.81 for the simpler model, on a 0-to-1 scale where higher is better).

Every number above is produced by the training run (`artifacts/cp1/report.md`), not typed by hand. Training takes under a minute on a normal laptop CPU.

6. Fairness and safety

- Gender, age, disability and similar details are **never** given to the model.
- We still measure fairness: false alarms are rare in every group (about 1-2%), but they differ by about 2 times between some groups, and younger students' dropouts are missed more often. We report this openly.
- The model only recommends. A person reviews every signal; nothing happens automatically.

7. Honest limits

- **No model catches everyone.** About 1 in 20 HIGH flags is a false alarm, and some at-risk students are not flagged. That is why a human reviews, and why the WATCH list exists.
- **The data is from Portugal.** It proves the system works end to end. Accuracy at an Indian college must be measured on that college's own data; the same code can retrain on it.
- **No attendance in this dataset.** Attendance is handled by simple rules today; with college data it can become a model input and give an even earlier warning.
- We left fees out on purpose (academic focus), although fees do help prediction in this dataset.

8. Questions judges may ask

Q: Is this just "attendance below 75%"?

No. Simple rules give readable reasons, but the main score combines many signs at once (pass rate, skipped exams, grades, trend, prior marks, course) and is tuned to real outcomes. It beats a rules-only approach.

Q: How do you stop false alarms?

We only alert when at least 85% of such students really are at risk, proven on unseen students. Medium cases need a rule to agree, and signals are capped at what mentors can handle.

Q: Could it cheat by using end-of-year results?

No. Each checkpoint only sees data available at that time, and our tests fail if future data leaks in.

Q: Is it biased?

Gender, age and disability are never inputs. We measure false alarms per group and report the gaps openly.

Q: Why should a mentor trust it?

Every signal shows checkable facts ("passed 2 of 6") and the main factors, and a person makes the final call.

Q: Will it work at an Indian college?

The same pipeline retrains on a college's data through a simple column mapping, and it must pass the same release check there before it is used.

Q: Why not use a neural network?

For a few thousand rows of table data, gradient boosting is usually at least as accurate, and much easier to explain. It trains in seconds on a normal CPU.

9. Words you might hear

Word	Simple meaning
Precision	Of the students we flagged, how many really were at risk (fewer false alarms = higher)
Recall	Of all at-risk students, how many we flagged (fewer missed = higher)
Calibration	Making the percentage match reality: 80% means about 8 in 10
Cutoff / threshold	The score above which we raise a signal
Train / test split	Learn from 80% of students; check on the other 20% the model never saw
Data leakage	Accidentally using future information, which makes a model look better than it is
Gradient boosting	Many small decision trees, each one fixing the last one's mistakes
Checkpoint	A moment in time (admission, after sem 1, after sem 2) with its own model

More detail: `ml/README.md` and `blueprint/16-risk-signal-pipeline.md` in the repository.